

Herald — MVP Specification V2

Herald — MVP Specification V2

Field	Value
Revision	1
Created	2026-05-19
Last modified	2026-05-19
Status	active
Status summary	V2 spec — TBDs populated, recommendations integrated, research-derived architecture decisions captured
Issues	none
Issues summary	implementation work tracked separately under workable-item prefix HRD– once scaffolding starts
Fixed	supersedes V1 — all V1 R-NN recommendations either applied or explicitly deferred to a named follow-up
Fixed summary	bi-directional event fan-out architecture; CloudEvents wire format; Watermill routing; Postgres + River queue (NATS opt-in); Apprise-style URL+tag channel model; Knock-style preference matrix; OpenTelemetry observability; Postgres RLS multi-tenancy; SLSA L3 supply chain; 9 named flavors (was 2 with TBD); per-channel rich-messaging feature matrix; email deep dive with DKIM + DMARC + RFC 8058 one-click unsubscribe
Continuation	implementation phases per §"Roadmap"; V1 (specification.V1.md) preserved as historical record and source of R-NN ID mapping

The **bi-directional event fan-out** system: Herald ingests events from heterogeneous sources and reliably fans them out to multiple notification channels so every alert reaches the right destination without confusion, and processes inbound replies/commands back from subscribers in a structured, security-validated way.

Table of contents

- [§1. Upstreams](#)
- [§2. Mission and scope](#)
 - [2.1 What Herald is](#)
 - [2.2 What Herald is NOT](#)

- 2.3 Architecture diagram
- §3. Execution model
 - 3.1 Single binary, two modes
 - 3.2 Distribution + invocation surfaces
 - 3.3 Configuration & credentials
- §4. Event model & wire format
 - 4.1 CloudEvents v1.0 as canonical envelope
 - 4.2 Herald event-type taxonomy
 - 4.3 Idempotency keys
- §5. Architecture overview
 - 5.1 Components
 - 5.2 Internal routing (Watermill)
 - 5.3 Queue backend (Postgres + River default, NATS opt-in)
 - 5.4 Retries & dead-lettering
 - 5.5 Webhook ingestion
 - 5.6 Orchestration of long-running operations (Temporal, opt-in)
- §6. Channel addressing & routing
 - 6.1 URL-scheme channel addresses (Apprise-style)
 - 6.2 Tag-based fan-out
 - 6.3 HeraldBranding (per-flavor visual identity)
- §7. Subscriber model
 - 7.1 Identity & reconciliation (per-channel-id + operator alias)
 - 7.2 Preferences (Knock-style PreferenceSet)
 - 7.3 Quiet hours & throttling
 - 7.4 Locale (i18n)
- §8. Workable-item naming prefix
 - 8.1 Static prefix HRD-
 - 8.2 Derived 3-letter prefix algorithm
- §9. Technology stack
 - 9.1 Go (single-binary, multi-flavor)
 - 9.2 Postgres + Row-Level Security
 - 9.3 Redis (per-tenant ACL)
 - 9.4 Container ports (24XXX)
 - 9.5 containers submodule
- §10. Commons (architecture layers)
- §11. Channels — per-channel capabilities matrix
 - 11.1 Telegram
 - 11.2 Max (max.ru)
 - 11.3 Slack
 - 11.4 Discord
 - 11.5 Microsoft Teams
 - 11.6 Lark / Feishu
 - 11.7 WhatsApp
 - 11.8 Viber
 - 11.9 Email (deep)
 - 11.10 ntfy / Gotify
 - 11.11 Generic outbound webhook
 - 11.12 Diary (Markdown + PDF + HTML)
 - 11.13 Feature matrix summary
- §12. Messaging flows
- §13. Templating & message composition
- §14. Localization (i18n)
- §15. Security model
 - 15.1 Transport-level (channel signature verification)

- [15.2 Sender-level \(allowlist + verified subscribers\)](#)
 - [15.3 Content-level \(parsing & sanitization\)](#)
 - [15.4 Credential handling](#)
 - [15.5 Webhook ingestion \(defense-in-depth\)](#)
 - [§16. Multi-tenancy & isolation](#)
 - [§17. Observability & SLOs](#)
 - [17.1 OpenTelemetry pipeline](#)
 - [17.2 Metrics catalogue](#)
 - [17.3 Span model](#)
 - [17.4 SLOs](#)
 - [17.5 Health probes \(livez / readyz / startupz\)](#)
 - [17.6 doctor CLI](#)
 - [§18. Flavors \(the implementations\)](#)
 - [18.1 Common flavor contract](#)
 - [18.2 Project Herald \(pherald\)](#)
 - [18.3 System Herald \(sherald\)](#)
 - [18.4 Build Herald \(bherald\)](#)
 - [18.5 Deploy Herald \(dherald\)](#)
 - [18.6 Alert Herald \(aherald\)](#)
 - [18.7 Schedule Herald \(scherald\)](#)
 - [18.8 Incident Herald \(iherald\)](#)
 - [18.9 Release Herald \(rherald\)](#)
 - [18.10 Compliance Herald \(cherald\)](#)
 - [18.11 Future flavors](#)
 - [§19. Diary](#)
 - [§20. Extensibility](#)
 - [§21. Supply chain & release engineering](#)
 - [§22. Constitution integration](#)
 - [§23. Specification documents \(change rule\)](#)
 - [§24. Documentation](#)
 - [§25. Testing](#)
 - [§26. Operations](#)
 - [§27. Roadmap](#)
 - [§28. Notes & open questions](#)
 - [§29. Changelog \(V1 → V2\)](#)
-

§1. Upstreams

All existing project upstreams:

- **GitHub** (main repository): `git@github.com:vasic-digital/Herald.git`
- **GitLab**: `git@gitlab.com:vasic-digital/herald.git`
- **GitFlic**: `git@gitflic.ru:vasic-digital/herald.git`
- **GitVerse**: `git@gitverse.ru:vasic-digital/Herald.git`

The local origin remote is a fan-out (one fetch URL + four push URLs). A single `git push origin <branch>` propagates to all four hosts (Helix Constitution §2.1 / Herald Constitution §103).

§2. Mission and scope

2.1 What Herald is

Herald is the **bi-directional event fan-out mechanism**. It receives input from one or more **sources** and dispatches the resulting content to one or more **destinations** (channels). Depending on the implementation (Flavor) of Herald, sources and destinations are heterogeneous: a single input type or many; a single output channel or many.

For example, the input may be the result of a CI pipeline execution (a build report, a test summary, a security-scan finding). Herald enriches, normalizes, routes, and dispatches that input to messaging channels (Telegram, Slack, Max, Email, Markdown diary, etc.) so that human and machine subscribers can be informed and can interact back.

The possibilities are not limited. The structure of the system **MUST** be **hierarchical**: shared abstractions live in the **commons** layers (closest to the root); flavor-specific implementations live in `flavors/<flavor>/`.

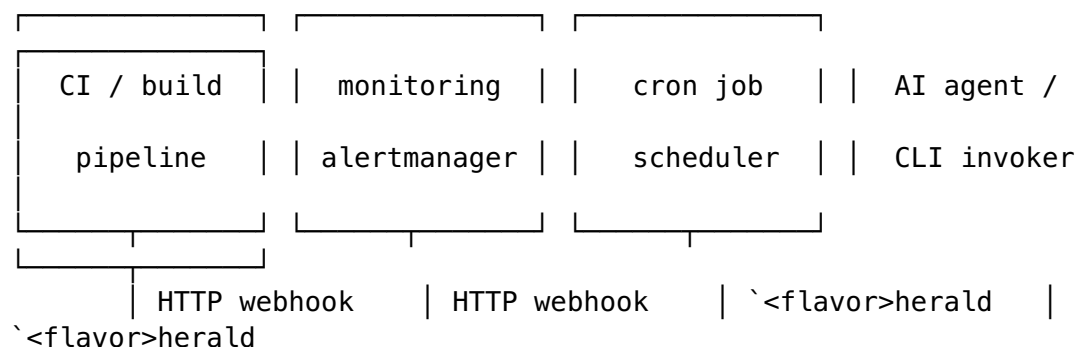
Note: We **MUST NOT** be obligated to follow this hierarchical structure rigidly when a parent project's specific custom flavor must exist privately or in a different location. Flexibility is mandatory and fully supported.

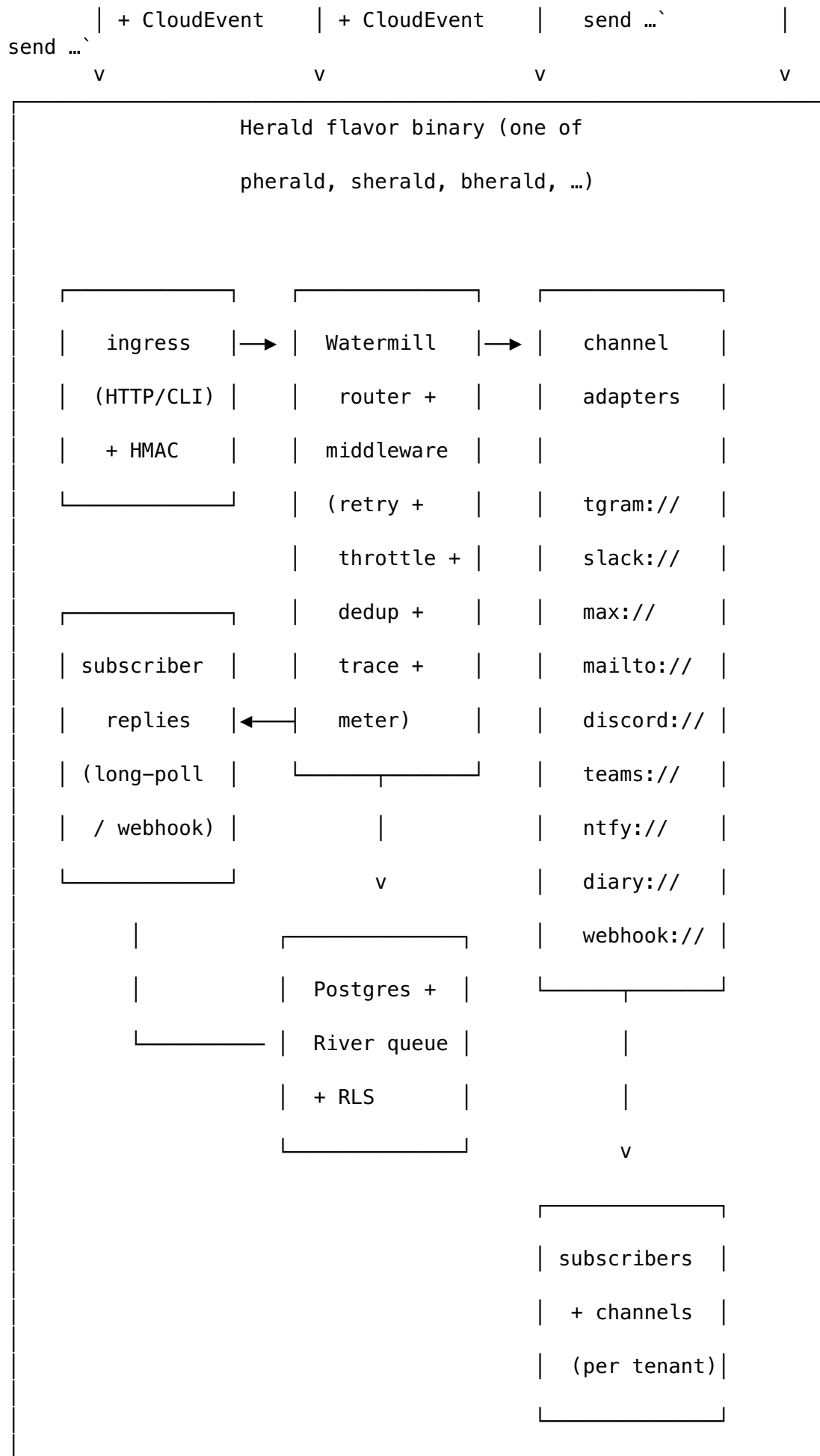
2.2 What Herald is NOT

To bound scope (§102 Herald Constitution — mission boundary):

- Herald is **not** a general-purpose chat application; it is an event-driven fan-out + inbound-reply processor.
- Herald is **not** a general-purpose monitoring/observability platform — it integrates *with* them via webhook ingestion (Prometheus Alertmanager, Grafana, Datadog, PagerDuty, OpsGenie) but does not replace them.
- Herald is **not** a transactional/marketing email service provider in itself — it integrates *with* ESPs (SendGrid, Resend, Postmark) as one of its delivery transports.
- Herald is **not** a workflow orchestration platform — Temporal/Argo/Step Functions remain the right tool for that; Herald can be triggered by them and can dispatch notifications about them.

2.3 Architecture diagram





|
v

```
docs/herald/diary/main.{md,pdf,html}
(every in/out message appended)
```

§3. Execution model

3.1 Single binary, two modes

Every Herald flavor is a **single statically-linked Go binary** with **Cobra-style subcommands** (per R-02 — cleanest fit, lowest deployment footprint, no duplicate auth/config plumbing). The same binary serves both runtime modes:

- **One-shot mode** — `<flavor>herald send ...`: connects, transmits a single event, awaits delivery acknowledgement with a configurable deadline, exits non-zero on failure. Designed for CI integration, cron jobs, AI-agent invocation, pipeline steps.
- **Daemon mode** — `<flavor>herald serve`: long-running listener; blocks on (a) the HTTP ingress for webhook events, (b) per-channel subscriber-reply loops (Telegram long-poll, Slack Socket Mode, Email IMAP, Discord gateway, etc.), (c) scheduled jobs from the River queue.

Both modes share the same configuration loader, channel adapters, storage layer, and observability stack — only the entry point and process lifecycle differ.

Additional subcommands:

- `<flavor>herald doctor` — verifies environment health (Postgres ping, Redis ping, channel-credential validity, DKIM/SPF/DMARC records, port availability).
- `<flavor>herald migrate` — applies database migrations (idempotent).
- `<flavor>herald deadletter [list | replay <id> | purge]` — operate on the dead-letter table.
- `<flavor>herald subscriber [list | add | verify <token>]` — manage subscribers.
- `<flavor>herald digest ...` — generate scheduled summaries (digests, weekly reports).

3.2 Distribution + invocation surfaces

Herald applications are CLI binaries primarily designed for:

- **CI integration** — invoked from GitHub Actions / GitLab CI / Jenkins / CircleCI / Drone steps.
- **Pipelines** — invoked from build/deploy scripts.
- **AI CLI agents** — invoked by Claude Code, OpenCode, Cursor, Aider, etc., as a structured way to surface progress and accept human feedback.
- **cron** — scheduled checks and digests.

- **Webhook recipients** — running in daemon mode behind a reverse proxy.

Some Herald application names (illustrative — flavors are fully enumerated in §18):

- pherald for **Project Herald** (development-lifecycle events).
- sherald for **System Herald** (OS/host/service events).
- bherald for **Build Herald** (CI/CD build events).
- dherald for **Deploy Herald** (release/deploy events).
- aherald for **Alert Herald** (monitoring alert routing).
- scherald for **Schedule Herald** (cron/scheduled-task notifications).
- iherald for **Incident Herald** (incident management).
- rherald for **Release Herald** (release lifecycle).
- cherald for **Compliance Herald** (audit/compliance events).

3.3 Configuration & credentials

Configuration precedence (12-factor aligned, per R-04):

1. Explicit CLI flag (highest precedence).
2. Shell-exported environment variable (from `.bashrc` / `.zshrc` / `k8s env` / `Docker -e`).
3. Value from `.env` (fallback only — does NOT override exported shell vars).
4. Compiled default (lowest).

`.env` MUST BE git-ignored. `.env.example` is the documented template, committed. An opt-in `--env-file-override` flag maps to `godotenv.Overload()` for the rare case operators want the file to win (one-off rescues, debugging).

Configuration file layout (`config.toml`, optional, loaded after `.env`):

```
[herald]
flavor      = "pherald"
tenant_id   = "00000000-0000-0000-0000-000000000001" # default
              if multi-tenancy disabled
locale      = "en-US"
diary_root  = "" # blank →
              discover via parent-walk
admin_port  = 24090 # /
              livez, /readyz, /metrics, pprof
http_port   = 24091 # webhook
              ingress + reply webhooks

[postgres]
dsn          = "${HERALD_POSTGRES_DSN}" # env
              interpolation
max_conns    = 20
rls_enforced = true

[redis]
addr         = "${HERALD_REDIS_ADDR}"
acl_user     = "${HERALD_REDIS_USER}"
acl_password = "${HERALD_REDIS_PASSWORD}"

[channels]
```

```
# Apprise-style URLs (see §6.1)
default = [
    "tgram://${TELEGRAM_BOT_TOKEN}/${TELEGRAM_CHAT_ID}?
        tags=oncall,prod",
    "slack://${SLACK_TOKEN}/${SLACK_CHANNEL}?tags=oncall",
    "mailto://herald@example.com?tags=audit",
    "diary:///tags=*", # always log to diary
]

[observability]
otlp_endpoint = "http://otel-collector:4317"
log_level      = "info"
```

§4. Event model & wire format

4.1 CloudEvents v1.0 as canonical envelope

Herald speaks **CloudEvents v1.0** (CNCF graduated, Jan 2024) natively on every external boundary — ingress, internal transport, audit/diary record. Inside the binary, events are passed as `cloudevents.Event` values from `github.com/cloudevents/sdk-go/v2`.

Required CloudEvents attributes for every Herald event:

Attribute	Source	Example
specversion	constant "1.0"	"1.0"
id	UUIDv7 (natural ordering by time)	"01931a7c-3f4e-7000-9abc- def012345678"
source	URI of the emitter	"https://ci.example.com/ pipelines/4231"
type	reverse-DNS event type	"digital.vasic.herald.ci.failed"
time	RFC 3339 timestamp	"2026-05-19T17:30:00Z"
datacontenttype	media type of data	"application/json"
subject	target hint (channel address, tag, route key)	"tag:oncall,prod" or "channel:slack-incidents"

Two wire modes accepted on HTTP ingress:

- **Structured mode** — single JSON object:
`{"specversion":"1.0","id":"...","type":"...","data":{"..."}}`.

- **Binary mode** — HTTP headers carry attributes (ce-id, ce-type, ce-source, ...), body is the raw data payload.

Herald-specific extension attributes:

- heralddenant — tenant UUID (overrides default).
- heraldidempotencykey — explicit dedup key (overrides id if present).
- heraldpriority — low / normal / high / urgent (ntfy-style 1–5 mapping).

4.2 Herald event-type taxonomy

CloudEvents type is a reverse-DNS string. Herald's namespace is `digital.vasic.herald.*`.

Subtree	Used by	Examples
<code>digital.vasic.herald.ci.*</code>	bherald	<code>.build.queued</code> , <code>.build.started</code> , <code>.build.completed</code>
<code>digital.vasic.herald.project.*</code>	pherald	<code>.task.opened</code> , <code>.task.assigned</code> , <code>.task.closed</code>
<code>digital.vasic.herald.system.*</code>	sherald	<code>.host.cpu_high</code> , <code>.host.disk_full</code> , <code>.service.restart</code>
<code>digital.vasic.herald.deploy.*</code>	dherald	<code>.deploy.started</code> , <code>.deploy.succeeded</code> , <code>.deploy.failed</code>
<code>digital.vasic.herald.alert.*</code>	aherald	<code>.alert.firing</code> , <code>.alert.resolved</code> , <code>.alert.acknowledged</code>
<code>digital.vasic.herald.schedule.*</code>	scherald	<code>.job.started</code> , <code>.job.succeeded</code> , <code>.job.failed</code>
<code>digital.vasic.herald.incident.*</code>	iherald	<code>.incident.opened</code> , <code>.incident.escalated</code> , <code>.incident.resolved</code>
<code>digital.vasic.herald.release.*</code>	rherald	<code>.release.tagged</code> , <code>.release.published</code> , <code>.release.retracted</code>
<code>digital.vasic.herald.compliance.*</code>	cherald	<code>.audit.access</code> , <code>.policy.violation</code> , <code>.certification</code>
<code>digital.vasic.herald.reply.*</code>	all	<code>.reply.received</code> , <code>.command.parsed</code> , <code>.command.executed</code>
<code>digital.vasic.herald.delivery.*</code>	internal	<code>.delivery.accepted</code> , <code>.delivery.routed</code> , <code>.delivery.failed</code>

4.3 Idempotency keys

Per R-05 + research Topic 4: **at-least-once delivery with deterministic dedup**.

- Every ingress accepts an Herald-Idempotency-Key HTTP header (or heraldidempotencykey CE extension, or CLI flag `--idempotency-key`).
- If absent, Herald falls back to the CloudEvents id.
- Dedup is enforced via a Postgres `idempotency_keys` table:

```
CREATE TABLE idempotency_keys (
  tenant_id      UUID NOT NULL,
  idempotency_key TEXT NOT NULL,
  request_hash   BYTEA NOT NULL,           -- SHA-256 of
  canonical_request_body
  response_id    UUID,                     -- pointer to
  first_response
  locked_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at     TIMESTAMPTZ NOT NULL DEFAULT (now() +
  interval '24 hours'),
  PRIMARY KEY (tenant_id, idempotency_key)
) WITH (FILLFACTOR = 80);
```

```
ALTER TABLE idempotency_keys ENABLE ROW LEVEL SECURITY;  
ALTER TABLE idempotency_keys FORCE ROW LEVEL SECURITY;  
CREATE POLICY idem_isolation ON idempotency_keys  
    USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

- Dedup INSERT ... ON CONFLICT DO NOTHING is in the **same transaction** as the channel-fanout enqueue. Stripe-pattern (cite: Brandur's writeup).
 - Redis is used as a **fast-path negative cache** (SET NX EX <ttl>) in front of Postgres, never as sole authority.
 - TTL: 24h for synchronous API replies (matches Stripe); 7d for delivery-side dedup (matches reply-queue retention).
 - Replay (same key, same body) returns the cached response; replay (same key, *different* body) returns HTTP 409 Conflict.
-

§5. Architecture overview

5.1 Components

A Herald flavor binary is composed of these in-process components:

1. **Ingress layer** — HTTP server (CloudEvents binding) + CLI command parser. Verifies signatures (§15.5), enforces idempotency (§4.3), and emits a Watermill message.
2. **Watermill router** — central message bus. Channel adapters, evaluators, and middleware (retry, dedup, throttle, trace, meter) are registered as `HandlerFuncs` on a single router.
3. **Channel adapters** — one per supported channel; implement the `Channel` interface (§10).
4. **Subscriber-reply consumers** — per-channel inbound loops (Telegram `getUpdates` long-poll or webhook, Slack Socket Mode / Events API webhook, Email IMAP, Discord gateway, etc.).
5. **Storage layer** — Postgres (durable state, RLS-isolated per tenant) + Redis (rate-limit token buckets, fast-path dedup, transient state).
6. **Queue backend** — Postgres + River as default; NATS JetStream as opt-in for multi-replica fan-out.
7. **Observability layer** — OpenTelemetry SDK (traces + metrics + logs), OTLP exporter, `/metrics` for Prometheus scrape, `/livez` / `/readyz` / `/startupz` probes.

5.2 Internal routing (Watermill)

Per research Topic 2: **Watermill** (github.com/ThreeDotsLabs/watermill, ~8.5k stars, v1.4+) is the routing kernel inside `commons_messaging`.

- Each channel adapter (Telegram, Slack, ...) is registered on a single `message.Router` as a `HandlerFunc`.
- Cross-cutting concerns (correlation ID, retry, poison-queue, metrics, throttling, tracing) are implemented as **Watermill middlewares** rather than per-channel code — eliminating ~60% of the plumbing the team would otherwise write.
- Pubsub backend is **pluggable**: Postgres adapter (default), NATS adapter (opt-in), in-memory adapter (tests).

5.3 Queue backend (Postgres + River default, NATS opt-in)

Per research Topic 3:

- **Default: Postgres + `riverqueue/river`** — transactional enqueue (jobs only run if the originating tx commits), built-in retries with backoff, uniqueness constraints. Zero new infrastructure since Postgres is already required.
- **`LISTEN/NOTIFY`** wakes consumers sub-millisecond between polls.
- **Opt-in alternative: NATS JetStream** — for deployments needing fan-out across multiple Herald instances, edge sites, or sub-millisecond cross-host latency (~200k–400k msg/s with persistence, built-in KV/Object stores).
- **Kafka is intentionally NOT the default** — overkill for Herald's alert-volume profile (dozens to low-thousands of events/sec per tenant). Document as Watermill-pluggable for operators who already run Kafka.

5.4 Retries & dead-lettering

Per research Topic 5 and AWS arch blog "Exponential Backoff and Jitter":

- **Backoff curve: decorrelated jitter** — `sleep = min(cap, random_between(base, prev_sleep * 3))` with `base = 1s`, `cap = 5min`.
- **Max attempts**: 8 per message for transient errors. Empirically: ~30min worst-case retry window, matching what humans expect of an alert system.
- **No retries on 4xx** from upstream channels — `400 / 401 / 403 / 404` from Telegram/Slack/etc. are permanent.
- **Implementation**: Watermill Retry middleware in front of channel handlers.
- **Dead letter sink: Postgres `dead_letters` table** (NOT a Redis list — operators need to query, triage, replay):

```
CREATE TABLE dead_letters (  
  id                UUID PRIMARY KEY DEFAULT uuidv7(),  
  tenant_id         UUID NOT NULL,  
  original_event_id UUID NOT NULL,  
  channel           TEXT NOT NULL,  
  attempt_count     INTEGER NOT NULL,  
  last_error        TEXT,  
  payload_jsonb     JSONB NOT NULL,  
  created_at        TIMESTAMPTZ NOT NULL DEFAULT now(),  
  triaged_at        TIMESTAMPTZ,  
  triage_status     TEXT -- new | acknowledged | replayed |  
                        discarded  
);  
ALTER TABLE dead_letters ENABLE ROW LEVEL SECURITY;  
ALTER TABLE dead_letters FORCE ROW LEVEL SECURITY;  
CREATE POLICY dl_isolation ON dead_letters  
  USING (tenant_id = current_setting('app.tenant_id')::uuid);
```

- Every entry into `dead_letters` also emits a `digital.vasic.herald.delivery.dead_lettered` CloudEvent — Herald observes its own failures.
- CLI: `<flavor>herald deadletter list | replay <id> | purge`.

5.5 Webhook ingestion

Per R-16 + research Topic 8. Every webhook source registers a per-source signing secret (rotatable) in the `webhook_sources` table.

The ingress handler enforces, in order:

1. **HMAC-SHA256 signature verification** with `crypto/hmac.Equal` (constant-time) against the source-configured header:
 - GitHub: `X-Hub-Signature-256`
 - Stripe: `Stripe-Signature` (sign `timestamp.payload`)
 - Generic CloudEvents source: `Webhook-Signature`
2. **Timestamp window** ≤ 5 min vs. server clock (replay protection).
3. **Delivery-ID dedup** via the idempotency table (GitHub's `X-GitHub-Delivery`, Stripe's event id).
4. **Optional IP allowlist** as L4 defense-in-depth.

Verified payloads are normalized into a `CloudEvent` and handed to the Watermill router.

Dev / behind-NAT support: `smee.io` and `gh webhook forward` are documented as supported proxy paths.

5.6 Orchestration of long-running operations (Temporal, opt-in)

Per research Topic 7:

- **Default routing** — Watermill handlers + River jobs cover the vast majority of fan-out cases.
 - **Temporal sidecar (opt-in)** — required for operations that genuinely need durable state across hours/days, deterministic replay, and human-friendly timeline UIs:
 - **Incident escalation chains** (page A → wait 5min → escalate to B → wait 10min → page C). Used by `iherald`.
 - **Scheduled digest builders** with human-in-the-loop acknowledgment.
 - **Multi-channel orchestrated rollouts** (used by `dherald` / `rherald`).
 - Temporal is explicitly **not** the default — its operational footprint (separate cluster, gRPC, server upgrades) violates Herald's "lightweight CLI" ethos.
-

§6. Channel addressing & routing

6.1 URL-scheme channel addresses (Apprise-style)

Per research Topic 1 (notification-platforms): adopt Apprise's URL-scheme channel model. Each destination is a single string that fully captures credentials + endpoint + targeting:

```
tgram://<bot_token>/<chat_id>?tags=oncall,prod
slack://<token_a>/<token_b>/<token_c>/<channel>?tags=oncall
max://<bot_token>/<chat_id>?tags=ru,prod
mailto://<user>:<pass>@<smtp_host>:<port>?
```

```
tags=audit&from=herald@example.com
discord://<webhook_id>/<webhook_token>?tags=community
teams://<incoming_webhook_path>?tags=corp
lark://<app_id>:<app_secret>@<chat_id>?tags=apac
ntfy://<server>/<topic>?priority=high&tags=mobile
gotify://<token>@<server>?priority=5
webhook://<endpoint_url>?secret=<hmac_secret>&format=cloudevent
diary:///tags=*&format=markdown
```

This URL form maps 1:1 to a row in Postgres `channel_addresses` and lets credentials be `.env`-interpolated at config-load time so the URLs themselves stay git-safe in committed `config.toml`.

6.2 Tag-based fan-out

Per Apprise's tag model: every channel address is annotated with one or more **tags**. Senders address tags, not specific channels:

- **OR-union by default:** `--tags=oncall,prod` matches any channel tagged `oncall` OR `prod`.
- **AND-intersection** when explicitly comma-joined inside a single argument: `--tags=oncall+prod` matches channels tagged `oncall` AND `prod`.
- Tag `*` is the wildcard (matches every channel).
- Tag `!oncall` means "exclude oncall" (intersect with negation).

Tag→channel mapping lives in the `channel_addresses` table; senders never reference channel addresses directly. This decouples sender code from topology — the same event can fan out to a different mix of channels per environment without code change.

6.3 HeraldBranding (per-flavor visual identity)

Per Apprise's `AppriseAsset`: a `HeraldBranding` struct injected per-flavor:

```
type Branding struct {
    AppName      string // "Project Herald"
    BinaryName   string // "pherald"
    IconURL      string // for rich embeds
    AccentColorHex string // "#2C7BE5"
    DefaultFooter string // "Sent by pherald 1.0 · github.com/vasic-digital/Herald"
}
```

Channel adapters consult `Branding` when rendering rich messages (Slack Block Kit headers, Discord embed author, Adaptive Card Container accent color, Email From-name).

§7. Subscriber model

7.1 Identity & reconciliation (per-channel-id + operator alias)

Per R-07 + research Topic 3 (notification-platforms): the **matterbridge model** is the default — per-channel-id is the source of truth. A subscriber on Telegram is identified by (channel:"tgram", chat_id:"42"); the same human on Slack is (channel:"slack", user_id:"U0123"); these are **separate entries** unless explicitly linked.

Schema:

```
CREATE TABLE subscribers (  
  id          UUID PRIMARY KEY DEFAULT uuidv7(),  
  tenant_id   UUID NOT NULL,  
  handle      TEXT,                      -- operator-assigned, e.g.  
             "alice"  
  display_name TEXT,  
  locale      TEXT DEFAULT 'en-US',  
  timezone    TEXT DEFAULT 'UTC',  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
ALTER TABLE subscribers ENABLE ROW LEVEL SECURITY;  
ALTER TABLE subscribers FORCE ROW LEVEL SECURITY;  
CREATE POLICY sub_isolation ON subscribers  
  USING (tenant_id = current_setting('app.tenant_id')::uuid);  
  
CREATE TABLE subscriber_aliases (  
  subscriber_id UUID NOT NULL REFERENCES subscribers(id) ON  
               DELETE CASCADE,  
  channel       TEXT NOT NULL,  
  channel_user_id TEXT NOT NULL,  
  verified_at   TIMESTAMPTZ,  
               -- self-claim verification time  
  UNIQUE (channel, channel_user_id)  
);
```

Linking flows:

- **Operator-mapped** (default): operator runs `pherald subscriber link --handle alice --tgram 42 --slack U0123`.
- **Self-claim** (opt-in per tenant): subscriber DMs a one-time token to the bot on each channel; the bot calls `pherald subscriber verify <token>` to record the alias.
- **Never inferred** — Herald does not guess identity across channels.

7.2 Preferences (Knock-style PreferenceSet)

Per research Topic 2 (notification-platforms): a Knock-inspired PreferenceSet JSON per subscriber:

```
{  
  "categories": {
```

```

    "incidents": { "channels": ["slack","tgram","email"],
    "muted": false },
    "deploys": { "channels": ["slack"], "muted": false },
    "daily_digest": { "channels": ["email"], "muted": false }
  },
  "workflows": {
    "digital.vasic.herald.alert.firing": { "muted": false,
    "channels": ["tgram","slack"] },
    "digital.vasic.herald.alert.resolved": { "muted": true }
  },
  "quiet_hours": { "tz": "Europe/Belgrade", "start": "22:00",
  "end": "07:00", "exempt_categories": ["incidents"] },
  "channel_data": {
    "tgram": { "chat_id": "42", "thread_id": null },
    "slack": { "user_id": "U0123", "im_channel": "D098" }
  }
}

```

- The router consults PreferenceSet before dispatching: if muted for the workflow/category, drop (and log to diary as "filtered"); otherwise restrict to listed channels and respect quiet hours.
- `channel_data` carries provider-specific routing keys that don't fit the generic alias model (Slack `im_channel` for DMs, Telegram `thread_id` for forum-topic posts).

7.3 Quiet hours & throttling

- **Quiet hours** — TZ-aware window per subscriber; exempt categories override.
- **Throttling** — per-(tenant, subscriber, category) token bucket in Redis (`herald:t:<id>:rl:<sub>:<cat>` with INCR+EXPIRE); default cap configurable per tenant.
- **Batching** — per category, optional `batch_window` (e.g. 5min); events within the window collapse into a single digest message. Implementation: River job scheduled `batch_window` after the first event arrives.

7.4 Locale (i18n)

Per research Topic 6 (notification-platforms): [nicksnyder/go-i18n v2](#) with CLDR plural rules.

- One Bundle loaded at process start from `commons_messaging/locales/*.toml`.
 - One Localizer per outgoing message, constructed from the subscriber's stored locale field (BCP-47 tag).
 - Per-channel templates can override (emoji-heavy Telegram template vs sober email template, both for the same locale).
 - Initial locales for V2: `en-US`, `sr-Latn-RS`, `ru-RU`. Additional locales as community contributes them.
-

§8. Workable-item naming prefix

8.1 Static prefix HRD–

For all opened workable items for the Herald project (Issues, Issues_Summary, Fixed, Fixed_Summary, Status, Status_Summary, etc.) use the prefix HRD. Examples: HRD-001, HRD-002. Strict zero-padded sequence within each docset.

8.2 Derived 3-letter prefix algorithm

Per R-17 — when a consuming project does NOT define its own workable-item prefix, Herald derives a deterministic 3-letter prefix from the project name. Implementation lives in `commons/prefix`.

Algorithm (~80 LOC):

1. **Normalize:** Unicode NFKD → strip diacritics → retain [A-Za-z0-9] → split on CamelCase boundaries and [-_ /] into tokens.
2. **Rule A (≥ 3 tokens):** first letter of each of the first three tokens. HeraldRouterCore → HRC.
3. **Rule B (2 tokens):** first letter of token 1; first letter of token 2; first internal consonant of token 2. HeraldRouter → HRT; HeraldRunner → HRN.
4. **Rule C (1 token):** first letter, first internal consonant, last consonant. Herald → HRD.
5. Uppercase.
6. **Collision resolution:** maintain committed `.herald/prefix.lock` (TOML name → `prefix`). On collision with a *different* project, compute `fnv1a32(name) mod 26` and replace the third letter with 'A' + (h mod 26); iterate up to 26 times, then fall back to numeric suffix HR0...HR9.
7. **Persistence:** the lock file is committed so the mapping is stable across machines and regenerations.

No mature Go library exists for 3-letter abbreviation generation (the only Go prior art Defacto2/releaser/initialism is a curated lookup table, not a generator); Herald ships its own.

§9. Technology stack

9.1 Go (single-binary, multi-flavor)

Herald and all flavors MUST BE written in **Go**. Layout (per R-18, research Topic 5):

```
Herald/
├─ go.work                # local dev only, .gitignored
├─ commons/               go.mod  # module github.com/vasic-digital/
herald/commons
├─ pherald/               go.mod  # module github.com/vasic-digital/
herald/pherald → cmd/pherald
├─ sherald/               go.mod  # module github.com/vasic-digital/
```

```

herald/sherald → cmd/sherald
├─ bherald/      go.mod    # ... etc
├─ .goreleaser.yaml      # one config, builds all flavors
├─ docs/, tests/, ...

```

Each flavor's go.mod declares require github.com/vasic-digital/herald/commons vX.Y.Z. **Lockstep release:** one git tag triggers GoReleaser to build all flavors and tag commons/vX.Y.Z. Use [release-please](#) in manifest mode to bump from Conventional Commits. go.work is git-ignored so CI catches forgotten bumps.

9.2 Postgres + Row-Level Security

Postgres is the main database, deployed via the containers submodule. Schema highlights:

- Every business table carries tenant_id UUID NOT NULL; composite indexes (tenant_id, ...).
- FORCE ROW LEVEL SECURITY enabled on every multi-tenant table.
- Runtime role: herald_app (no BYPASSRLS). Migration role: herald_migrator (BYPASSRLS).
- SET LOCAL app.tenant_id = '<uuid>' at the start of every transaction.
- UUIDv7 (uuidv7() function) for time-ordered primary keys.

9.3 Redis (per-tenant ACL)

Redis is the in-memory store, deployed via containers submodule.

- **Per-tenant ACL user:** redis-cli ACL SETUSER tenant_<id> on >pwd ~t:<id>:* +@read +@write ...
- **Key prefixing:** t:<tenant_id>:... on every key.
- Usage: rate-limit token buckets, fast-path idempotency-cache, transient deduplication, hot subscriber lookups.

9.4 Container ports (24XXX)

Per R-01: all Container host ports start with **24XXX** prefix (1024–49151 IANA User Ports, below the Linux ephemeral floor at 32768, above all common service defaults — Postgres 5432, Redis 6379, web 3000/5000/8000/8080/9000).

Reserved sub-blocks:

Range	Purpose
24000–24099	flavor app data ports (one per flavor instance)
24090–24099	flavor admin ports (/livez, /readyz, /metrics, pprof)
24100–24199	Postgres instances (default 24100)
24200–24299	Redis instances (default 24200)

Range	Purpose
24300–24399	NATS JetStream (when enabled)
24400–24499	OpenTelemetry Collector (default OTLP gRPC 24417, HTTP 24418)
24500–24599	Prometheus / Grafana (when self-hosted alongside)
24600–24699	Temporal (when enabled)
24700–24999	reserved for future / per-tenant

9.5 containers submodule

The full Docker/Podman Compose stack is provided by [vasic-digital/containers](#) as a Git submodule (per R-12, the owned-submodule set in `HERALD_CONSTITUTION.md` will be updated in the PR that introduces the submodule). All container names MUST start with prefix `herald`.

§10. Commons (architecture layers)

Per the layering principle in V1: commons -> commons level 1 -> ... -> commons level N -> Flavor. V2 names the layers concretely:

Layer	Module	Owns
L0	commons	CloudEvents envelope, Channel interface, Branding, error types, time/uuid helpers
L1	commons_messaging	Watermill router, retry/throttle/dedup middlewares, channel adapters (Telegram, Slack, ...), templating, i18n
L1	commons_storage	Postgres connection + RLS middleware, River queue setup, Redis client + tenant ACL, migrations
L1	commons_security	HMAC verifiers (Slack/GitHub/Telegram/generic), DKIM/SPF/DMARC helpers, allowlist evaluator, command parser
L1	commons_observability	OTel setup, span helpers, metrics registration, log handler wiring (<code>slog</code> + <code>otel slog</code>)
L1	commons_prefix	3-letter prefix algorithm (§8.2)
L1	commons_diary	append + export + sync (§19)
L2	commons_workflows	Temporal workflow scaffolding (escalation chains, digest builders) — opt-in
Flavor	pherald, sherald, ...	Per-flavor commands, event-type handlers, flavor-specific subscriber commands

Rule of thumb (carry from V1): put new shared code in the **lowest layer that still makes sense**; flavors inherit upward.

§11. Channels — per-channel capabilities matrix

Each channel adapter implements:

```
package commons // L0

type Channel interface {
    Name() string // "tgram",
                  "slack", ...
    Capabilities() Capabilities // see §11.13
    Send(ctx context.Context, msg OutboundMessage) (Receipt,
        error)
    Subscribe(ctx context.Context, h InboundHandler) error //
        long-running; called by `serve`
    HealthCheck(ctx context.Context) error
}
```

11.1 Telegram

- **SDK:** `gopkg.in/telebot.v3` (tucnak/telebot) primary; `github.com/mymmrac/telego` alt for 1:1 Bot API fidelity.
- **V1 (MVP) features:**
 - `sendMessage` with `MarkdownV2` parse mode.
 - `sendPhoto` / `sendDocument` for attachments.
 - `InlineKeyboardMarkup` with URL buttons + `callback_data`.
- **V2 advanced:**
 - `callback_query` handler (interactive replies).
 - `editMessageText` (in-place updates for ongoing incidents).
 - `web_app` buttons (open Mini-Apps).
 - `sendMediaGroup` (galleries).
 - Forum-topic threads via `message_thread_id`.
- **Out-of-scope (V2):** full Mini-App SDK, payments, Telegram Passport.
- **Delivery evidence ceiling:** Routed via Bot API; Read only via Business Bot connection with `can_read_messages` right (separate setup).
- **Webhook secret:** `setWebhook?secret_token=<token>`; verify `X-Telegram-Bot-API-Secret-Token` header on inbound.

11.2 Max (max.ru)

- **SDK:** `github.com/max-messenger/max-bot-api-client-go` (official-adjacent, Apache-2.0, English+Russian docs).
- **Gating:** behind build tag `herald_max` and documented sanctions advisory in the operator manual. VK Group parent VK Company Limited is not on the OFAC SDN list at time of writing, but several VK-affiliated individuals are designated and EU restrictive measures evolve frequently — operators must check at deploy time.
- **Bot registration:** via in-app MasterBot.
- **V1:** send text message, send media; basic inline keyboard.

- **V2 advanced:** per `dev.max.ru` docs as they expand.
- **Delivery evidence ceiling:** Routed (API ack = stored & queued).

11.3 Slack

- **SDK:** `github.com/slack-go/slack` (pin $\geq$ v0.23.1 — GHSA-gxhx-2686-5h9g fixed empty-secret bypass).
- **V1:**
 - `chat.postMessage` with Block Kit header + section + divider + image + actions (URL buttons only).
 - Threading via `thread_ts` (R-08 mapping).
- **V2 advanced:**
 - `block_actions` callback handler (interactivity).
 - `views.open` modals.
 - Message shortcuts.
 - Socket Mode for daemon ingress.
- **Out-of-scope:** Workflow Builder steps, Slack Connect, Enterprise Grid management.
- **Delivery evidence ceiling:** Routed (`ok:true` + `ts` proves posted to channel; no per-user read event).
- **Signing:** HMAC-SHA256 verification using `slack.NewSecretsVerifier` over `v0:{X-Slack-Request-Timestamp}:{rawBody}`, comparing with `X-Slack-Signature`, 5-min replay window.

11.4 Discord

- **SDK:** `github.com/bwmarrin/discordgo` (~5.7k stars, BSD-3-Clause, stable).
- **V1:**
 - Incoming Webhook with embeds (title, description, fields, footer, thumbnail, color).
 - Threads via `webhook + ?thread_id=`.
- **V2 advanced:**
 - Bot token; components (buttons + select menus).
 - Slash commands.
 - Forum channels.
- **Out-of-scope:** modals, voice, stage channels.
- **Delivery evidence ceiling:** Routed.

11.5 Microsoft Teams

- **SDK:** `github.com/atc0005/go-teams-notify/v2` (incoming-webhook-only — **no official Microsoft Go SDK** for full Graph access).
- **V1:**
 - Incoming Webhook with Adaptive Card v1.5: `TextBlock`, `Image`, `Container`, `FactSet`, `ActionSet` with `Action.OpenUrl`.
- **V2 advanced:**
 - `Action.Execute` / Actionable Messages with HMAC-signed tokens.
 - Bot Framework via Azure Bot Service (separate setup, complex).
- **Out-of-scope:** full Bot Framework conversational state, meeting extensions.
- **Delivery evidence ceiling:** Routed.

11.6 Lark / Feishu

- **SDK:** github.com/larksuite/oapi-sdk-go (**official**, MIT, code-gen flavor — verbose but accurate).
- **V1:** send text + rich message + image; basic interactive cards.
- **V2 advanced:** Mini-program cards, chatbot @-mentions, group management.
- **Delivery evidence ceiling:** Routed.

11.7 WhatsApp

Two transports for two use cases:

- **WhatsApp Web multi-device** — go.mau.fi/whatsmeow ([tulir/whatsmeow](https://tulir.com/whatsmeow), ~5.5k stars, MPL-2.0). Reverse-engineered WA Web; **not** for official Business API.
- **WhatsApp Business Cloud API** — github.com/twilio/twilio-go (official, multi-product; WA via Twilio Senders).
- **V1:** text + media via either transport; template-message support on Business Cloud API.
- **V2 advanced:** button-template messages, list messages, conversation pricing windows.

11.8 Viber

- **No official Go SDK.** Community: [mileusna/viber](https://mileusna.com/viber), [strongo/bots-api-viber](https://strongo.com/bots-api-viber).
- **V1:** hand-rolled REST client against the documented Viber REST API. Send text + media + carousel.
- **V2 advanced:** rich-media keyboards.

11.9 Email (deep)

The most operationally complex channel — Email gets the deepest treatment.

Two interchangeable transports behind one Channel interface:

1. **Raw SMTP + DKIM** — net/smtp (stdlib) + github.com/emersion/go-msgauth/dkim. Suitable for self-hosted operators.
2. **ESP HTTP API** — Resend (preferred), Postmark (fallback), SendGrid (alternative). Better deliverability + built-in bounce/complaint webhooks + suppression lists.

Mandatory features for every outbound email (V1):

- **DKIM signing** (RFC 6376) via [go-msgauth/dkim](https://go-msgauth.com/dkim/).
- **List-Unsubscribe** (RFC 8058) — `List-Unsubscribe: <https://...>`, `<mailto:...>` + `List-Unsubscribe-Post: List-Unsubscribe=One-Click`. DKIM-signed.
- **Plain-text alternative** generated from the HTML body (MJML-rendered).
- **Inline images** via `Content-ID: MIME parts (cid: references in HTML)`.
- **Suppression list lookup** — consult `email_suppressions` table before send; skip suppressed addresses and log to diary.

Bounce pipeline:

- Inbound mailbox parsed for RFC 3464 DSN parts, OR ESP webhook consumed for bounce/complaint/unsub events.
- Hard bounces, complaints, and manual unsubscribes write to email_suppressions:

```
CREATE TABLE email_suppressions (  
  tenant_id    UUID NOT NULL,  
  address      TEXT NOT NULL,           -- normalized lowercase  
  reason       TEXT NOT NULL,         -- 'hard_bounce' |  
    'complaint' | 'unsubscribe'  
  source_event UUID,  
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),  
  PRIMARY KEY (tenant_id, address)  
);  
ALTER TABLE email_suppressions ENABLE ROW LEVEL SECURITY;  
ALTER TABLE email_suppressions FORCE ROW LEVEL SECURITY;
```

Doctor command: <flavor>herald doctor email verifies SPF, DKIM, DMARC records for the configured sending domain at startup. Without correct DNS, Gmail/Yahoo's 2024 sender requirements quietly bin the mail.

Templating: MJML (<mj-section>, <mj-column>, <mj-image>, <mj-button>) compiled to inline-CSS HTML at **template-author time** (vendored MJML CLI; check in the compiled HTML alongside the .mjml source). The compiled HTML then runs through html/template for variable substitution at send time. No Node.js in the runtime container.

Delivery evidence ceiling: Accepted by default (relay 250 OK), upgradable to Delivered by parsing DSN, Read only when recipient voluntarily returns RFC 8098 MDN (rare).

11.10 ntfy / Gotify

Per research Topic 3 (notification-platforms): adopt ntfy's wire model as both an **inbound ingest format** (alongside CloudEvents) and as a first-class outbound channel.

- Map: X-Priority (1-5) → Herald priority enum; X-Tags → Herald tags; X-Click → action URL; X-Actions (view/http/broadcast/copy) → Herald Action schema; X-Attach/PUT body → attachment.
- **Gotify** application/client token split: application token authenticates publishers; client token authenticates subscribers; both revocable independently.
- **V1:** send to ntfy/gotify topic with priority + tags + attachment.
- **V2 advanced:** receive via WebSocket / SSE for live updates.

11.11 Generic outbound webhook

Adapter webhook: //<url>?secret=<hmac>&format=<cloudevent|json|form>:

- Sends Herald events as CloudEvents (binary or structured mode), or as plain JSON, or as form-encoded — chosen by query param.

- HMAC-signs outbound requests (Webhook-Signature header) so receiving side can verify.
- Retries per §5.4.

11.12 Diary (Markdown + PDF + HTML)

See §19 for full schema and sync strategy.

11.13 Feature matrix summary

Channel	Text	Markdown	Attachments	Threads	Interactive buttons	Signed-source verify
Telegram	✓	MarkdownV2 / HTML	✓	forum topic_id	✓ V1 url, V2 callback	secret_tok
Max	✓	platform-specific	✓	V2	V2	tba
Slack	✓	Block Kit	✓	thread_ts	✓ V1 url, V2 callback	X-Slack-Signature
Discord	✓	embeds	✓	thread_id	webhook url, bot components	webhook URL secrecy
MS Teams	✓	Adaptive Card	inline	conv references	OpenUrl / Action.Execute	webhook secret
Lark	✓	rich card	✓	V2	interactive card	event subscription
WhatsApp	✓	text/template	✓ (Cloud API)	conversation window	Cloud API templates	Twilio signing / Meta token
Viber	✓	rich-media	✓	—	rich-media keyboard	sig param
Email	✓	MJML→HTML	✓ (MIME)	In-Reply-To/References	url buttons only	DKIM/SPDMARC
ntfy	✓	X-Markdown	✓	—	X-Actions	basic auth token
Gotify	✓	extras.client::display	✓	—	extras actions	bearer token
Webhook	✓	CloudEvent JSON	embedded	n/a	n/a	HMAC
Diary	✓	source format	path-refs	logical via parent ref	n/a	local FS

§12. Messaging flows

Three mandatory message flows per channel (where the channel API supports them):

- **Simple message** — textual content sent to the channel; displayed to all subscribers.
- **Message with attachment(s)** — content + 1..N attachments; subscribers can download attachments.
- **Quote / reply message** — content sent as a reply to an existing channel message (uses §11 thread mapping: Telegram `reply_to_message_id`, Slack `thread_ts`, Discord `thread_id`, Email In-Reply-To); may include 0..N attachments.

Subscriber inbound flows (Project Herald and any flavor that opts into reply processing):

- **Direct reply to a Herald message** — parsed for commands (Bug:, Issue:, Query:, Request:, Question:).
- **Standalone message tagged for Herald** — same parsing, no parent message.
- **Thread continuation** — full thread context (chained replies from start) is parsed and processed.

Conversation reference per R-08:

```
type ConversationRef struct {
    Channel      ChannelID // "tgram", "slack", ...
    ThreadID     string    // Slack thread_ts; Telegram
                message_thread_id (forum); Email References[0]
    ParentMessageID string    // Slack ts; Telegram
                reply_to_message_id; Email In-Reply-To
    RootMessageID string    // first message in the thread
}
```

Persisted in `thread_refs` (`tenant_id`, `logical_thread_id`, `channel`, `ref_json`) so re-sends to the same logical thread find the right native handle on every channel.

§13. Templating & message composition

Three-layer stack per research Topic 5 (notification-platforms):

- **Layer 0 (engine)** — Go `stdlib` text/template (plaintext / Markdown / JSON channels) and `html/template` (HTML email body, auto-escaping).
- **Layer 1 (helpers)** — `Sprig` (date, upper, default, trunc, etc.) — universally expected by template authors; used by Helm, `kubectl` templates.
- **Layer 2 (email-specific)** — MJML compiled to HTML at template-author time (vendored MJML CLI; check in the compiled `.html` alongside `.mjml`), then `html/template` for runtime substitution.

Rejected: Liquid (extra runtime, no Go `stdlib` affinity); runtime MJML compilation (Node dependency in Go hot path).

Template directory layout:

```
commons_messaging/templates/
├── ci.failed/
│   ├── tgram.md.tpl
│   ├── slack.json.tpl      # Block Kit JSON
│   ├── email.mjml         # source
│   ├── email.html         # compiled, committed
│   └── diary.md.tpl
├── deploy.succeeded/
│   └── ...
├── shared/                 # template helpers
└── footer.tpl
```

§14. Localization (i18n)

Per research Topic 6 (notification-platforms): [nicksnyder/go-i18n v2](#).

- One Bundle loaded at process start from `commons_messaging/locales/*.toml`:

```
# locales/active.en-US.toml
[ci.failed.title]
other = "Build failed: {{.JobName}}"
```

```
# locales/active.sr-Latn-RS.toml
[ci.failed.title]
other = "Build pao: {{.JobName}}"
```

- Plurals handled via CLDR rules (built-in to `go-i18n`):
1 alert / 2 alerta / 5 alerti / 21 alert for Serbian works correctly without custom code.
- Per-subscriber locale (BCP-47 tag) stored on the subscribers row.
- Per-channel templates may override (Telegram emoji-heavy variant vs sober email variant).

V2 initial locales: `en-US`, `sr-Latn-RS`, `ru-RU`. RTL rendering tweaks and dynamic locale negotiation from incoming events are deferred.

§15. Security model

Per R-16: a **three-layer pipeline** runs *before* any routing logic.

15.1 Transport-level (channel signature verification)

Per-channel signature verifier (constant-time HMAC comparison via `crypto/hmac.Equal`):

Channel	Header	Algorithm	Replay window
Slack	X-Slack-Signature	HMAC-SHA256 over <code>v0:{ts}:{body}</code>	5 min
GitHub-style	X-Hub-Signature-256	HMAC-SHA256 over body	configurable
Telegram	X-Telegram-Bot-API-Secret-Token	shared-secret compare	n/a (no timestamp)
Stripe	Stripe-Signature	HMAC-SHA256 over <code>{ts}.{body}</code>	5 min
Email	DKIM-Signature header + DMARC alignment	RSA-SHA256 / Ed25519	per DKIM TTL

Generic webhook sources register their own per-source secret in `webhook_sources`.

15.2 Sender-level (allowlist + verified subscribers)

After transport verification, sender authority is checked:

- Per-channel allowlist keyed by canonical user-id (Slack `team_id+user_id`, Telegram `chat_id`, email DKIM-aligned From) loaded from `.herald/subscribers.toml` and/or the `subscribers + subscriber_aliases` tables.
- Unknown senders enter a **quarantine queue** (`quarantined_messages` table), never the live fan-out.
- An operator (or auto-policy) reviews the quarantine and either promotes the sender or discards.

15.3 Content-level (parsing & sanitization)

After sender verification, content is parsed:

- Commands (`Bug:`, `Issue:`, `Query:`, `Request:`, `Question:`, plus flavor-specific verbs) parsed with a strict regex-anchored tokenizer.
- **Never** shell out with user-provided content.
- **Never** text/template user input into shell or SQL.
- Command bodies are length-bounded (default 8 KiB).
- Attachments are scanned for MIME mismatch + size limits (default 25 MiB per attachment, 100 MiB per message).
- Optional malware scan via ClamAV when `commons_security` is configured with the integration.

15.4 Credential handling

Per Constitution §11.4.10:

- `.env` is git-ignored (the `.gitignore` MUST contain `.env`).
- `.env.example` is committed.
- Resolution order: CLI flag > shell env > `.env` > compiled default (§3.3).

- Credentials NEVER logged. The OTel logger has a redaction middleware that filters keys matching (?i)(token|secret|password|key|credential|authorization) to ***.
- Credentials in Postgres are encrypted-at-rest with pgcrypto (pgp_sym_encrypt) using a key from HERALD_DB_ENC_KEY env var.

15.5 Webhook ingestion (defense-in-depth)

See §5.5. Four ordered checks: signature → timestamp → delivery-ID dedup → optional IP allowlist.

§16. Multi-tenancy & isolation

Per research Topic 6 (event-fanout) + Topic 4 (operations):

- **Primary boundary:** PostgreSQL Row-Level Security.
- **Every business table** carries tenant_id UUID NOT NULL with a composite index (tenant_id, ...) leading every secondary index (RLS is 10–100× slower without it).
- **Policy template:**

```
ALTER TABLE <table> ENABLE ROW LEVEL SECURITY;
ALTER TABLE <table> FORCE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON <table>
    USING (tenant_id = current_setting('app.tenant_id')::uuid)
    WITH CHECK (tenant_id =
        current_setting('app.tenant_id')::uuid);
```

- **Roles:**
 - herald_app — runtime role, no BYPASSRLS. The pgx connection wrapper in commons_storage runs SET LOCAL app.tenant_id = ... at transaction start.
 - herald_migrator — schema-change role, BYPASSRLS.
 - **Redis** — per-tenant ACL user (Redis 6+), key pattern ~t:<tenant_id>:*
 - **Rate limits** — token bucket per (tenant_id, channel) in Redis.
 - **Schema-per-tenant** is reserved for high-isolation enterprise customers; documented as opt-in but not the default (overhead: connection-pool fragmentation, slow migrations $N \times M$, harder backups).
-

§17. Observability & SLOs

Per research Topic 1 (operations):

17.1 OpenTelemetry pipeline

- **All three signals** instrumented from day one:
 - **Traces** — go.opentelemetry.io/otel/trace spans across the full event flow (ingest → route → enqueue → deliver → ack).

- **Metrics** — `go.opentelemetry.io/otel/metric` counters + histograms.
- **Logs** — `stdlib slog` shipped as OTEL log records via `go.opentelemetry.io/contrib/bridges/otel slog`.
- **Default export:** OTLP/gRPC to an OpenTelemetry Collector sidecar/DaemonSet.
- **Collector** fans out to Prometheus (scrape `/metrics`), Tempo/Jaeger (traces), Loki (logs).
- Instrumentation lives in `commons_observability` (L1) so every flavor inherits.

17.2 Metrics catalogue

OTel semantic conventions for messaging where they fit, Herald-specific names where they don't:

Metric	Type	Labels
<code>messaging.client.sent.messages</code>	counter	<code>messaging.system="herald"</code> , <code>messaging.destination.name=<channel></code> , <code>herald.tenant_id</code> , <code>herald.flavor</code>
<code>messaging.client.operation.duration</code>	histogram	+ <code>messaging.operation.name="deliver"</code> , <code>messaging.operation.type="send"</code>
<code>herald_delivery_attempts_total</code>	counter	<code>channel</code> , <code>outcome={ok,retry,drop,dead_letter}</code> , <code>tenant_id</code>
<code>herald_queue_depth</code>	gauge	<code>queue</code> , <code>tenant_id</code>
<code>herald_retry_backoff_seconds</code>	histogram	<code>channel</code> , <code>attempt_bucket</code>
<code>herald_template_render_duration_seconds</code>	histogram	<code>template_id</code>
<code>herald_subscriber_count</code>	gauge	<code>tenant_id</code> , <code>channel</code>
<code>herald_dead_letter_count</code>	gauge	<code>tenant_id</code> , <code>channel</code>

Cardinality budget: never `recipient_id` or message UUIDs as labels. Per-tenant labels are allowed; per-subscriber labels are NOT.

Canonical Grafana dashboard JSON shipped in `deploy/grafana/herald-overview.json`.

17.3 Span model

Canonical span names (and parent→child relationship):

```

herald.ingest          (root, per inbound CloudEvent)
├── herald.signature_verify
├── herald.idempotency_check
├── herald.route_match
├── herald.preference_filter
├── herald.template_render
├── herald.enqueue
└── herald.deliver
    ├── herald.channel.<name>
    │   └── herald.channel.<name>.api_call
    └── herald.diary_append

```

Attributes on every span: `herald.tenant_id`, `herald.flavor`, `cloudevents.type`, `cloudevents.id`.

17.4 SLOs

V2 published SLOs (per-tenant rolling 30-day):

SLO	Target
Ingress availability (HTTP 2xx + 4xx vs 5xx)	99.9%
End-to-end delivery (ingest → channel API Accepted) — p95	< 5 s
End-to-end delivery — p99	< 30 s
Delivery success rate (excluding 4xx from upstream channels)	99.5%
Dead-letter rate	< 0.1%
Doctor-CLI checks (SPF/DKIM/DMARC, DB ping, Redis ping)	run every 5 min, alert on 3 consecutive fail

17.5 Health probes (livez / readyz / startupz)

Per research Topic 3 (operations): three endpoints on the **admin port** (24090 default, separate from data port so probes don't compete for worker capacity):

- **GET /livez** — returns **200** unless an unrecoverable internal invariant trips (deadlock detector, fatal goroutine panic flag). Cheap, no downstream calls.
- **GET /readyz** — checks Postgres + Redis ping, queue consumer attached, ≥ 1 channel adapter healthy. Returns **503** during graceful shutdown.
- **GET /startupz** — used by Kubernetes startup probe while migrations / warm-up run; once **200**, liveness/readiness take over.

Liveness failureThreshold $\sim 3\times$ readiness so a transient downstream blip removes the pod from Service but does NOT restart it.

17.6 doctor CLI

<flavor>herald doctor runs a battery of environment checks:

- Postgres connectivity + RLS policy presence.
- Redis connectivity + ACL.
- All configured channel credentials valid (probes API per channel).
- DKIM/SPF/DMARC DNS records for the configured sending domain (cite Gmail/Yahoo 2024 sender requirements).
- OTLP collector reachable.
- Disk space, port availability.

Exits with non-zero status code on any failure + writes a structured report to stdout (machine-readable JSON option `--json`).

§18. Flavors (the implementations)

18.1 Common flavor contract

Every flavor binary MUST:

- Be named `<prefix>herald` and built from `<prefix>herald/cmd/` `<prefix>herald/main.go`.
- Implement the same Cobra-subcommand surface (send, serve, doctor, migrate, deadletter, subscriber, digest, version).
- Embed `commons` + `commons_messaging` + `commons_storage` + `commons_security` + `commons_observability` + `commons_diary` at the same pinned versions.
- Publish a flavor-specific event-type subtree (`digital.vasic.herald.<flavor>.*`).
- Register flavor-specific subscriber commands (the Bug: / Query: / etc. tokens that map to flavor-specific handlers).
- Provide a flavor-specific HeraldBranding (app name, icon, accent color).
- Ship its own user manual under `docs/flavors/<flavor>/`.

18.2 Project Herald (pherald)

Focus: Software-project development lifecycle. Integrates with VCS, code review, task tracking. Designed for AI-CLI-agent + developer-team workflows.

Event subtree: `digital.vasic.herald.project.*` and `digital.vasic.herald.reply.*`.

Sources:

- Git hooks (post-commit, post-merge, pre-push).
- VCS webhooks (GitHub, GitLab, GitFlic, GitVerse — push, pull_request, review, issue).
- AI CLI agents (Claude Code, OpenCode, Cursor, Aider) emitting structured progress events.
- Code-review-tool webhooks (CodeRabbit, Greptile, Sourcery).

Subscriber inbound commands (extends the V1 base set):

Command	Behaviour
Bug: / Issue:	Open an issue (writes to <code>docs/Issues.md</code> per Universal §11.4.12).
Query: / Request: / Question:	Request information; routes to LLM-driven research workflow if configured.
Status:	Reply with current project status from <code>docs/Status.md</code> .
Continue:	Pointer to <code>docs/CONTINUATION.md</code> for next agent.
Done:	Mark referenced item resolved (writes to <code>docs/Fixed.md</code>).
Spec:	

Command	Behaviour
	Cite or modify spec section (subject to §23 spec-change rule).
Run: <tool>	Authorized operator only — execute an approved tool (gated by allowlist).

Quote-thread processing: Subscriber's reply automatically includes the full thread context (chained replies from bottom to thread start) — full chain parsed and processed per R-08 ConversationRef.

Security validation (extends §15):

- Subscriber commands accepted only from verified, allowlisted subscribers.
- Run: requires elevated subscriber privilege (subscriber_aliases.verified_at plus roles array).
- All commands logged to diary with subscriber identity and timestamp.

Derived workable-item prefix (per §8.2): if the consuming project hasn't set one, the algorithm runs over the project name (from package.json, go.mod, pyproject.toml, or the git remote name).

18.3 System Herald (sherald)

Focus: OS/host/service health and security. Designed for systems-administration teams + on-call rotation.

Event subtree: digital.vasic.herald.system.*.

Sources:

- journalctl watcher (configurable _SYSTEMD_UNIT filters).
- auditd events.
- Prometheus Alertmanager webhook receiver.
- Loki/Grafana log alerts.
- File-integrity-monitoring (AIDE, Tripwire) hooks.
- SNMP traps (via a small SNMP-to-CloudEvent adapter).
- Kernel events (oom-kill, panics) — via journald filter.

Event types:

- digital.vasic.herald.system.host.cpu_high (threshold-crossing).
- digital.vasic.herald.system.host.disk_full (filesystem).
- digital.vasic.herald.system.host.memory_pressure.
- digital.vasic.herald.system.service.restarted.
- digital.vasic.herald.system.service.crashed.
- digital.vasic.herald.system.service.flapping (≥ 3 restarts in 5min).
- digital.vasic.herald.system.security.login_anomaly (failed SSH burst, unusual IP).
- digital.vasic.herald.system.security.privilege_escalation.
- digital.vasic.herald.system.cert.expiring (X.509 certificate ≤ 30 d).
- digital.vasic.herald.system.backup.missed.

Subscriber commands:

- **Ack:** — acknowledge an alert (silences future fires of the same fingerprint for a window).
- **Silence:** <fingerprint> for <duration> — manual silencing.
- **Resolve:** <fingerprint> — manual resolution.
- **Status:** — current open-alerts snapshot.
- **Runbook:** <fingerprint> — link to the runbook for this alert type.

Integrations: sherald is often paired with aherald and iherald — the three flavors share the commons_alert package.

18.4 Build Herald (bherald)

Focus: CI/CD build lifecycle events. Designed for development teams + release engineers.

Event subtree: digital.vasic.herald.ci.*.

Sources:

- GitHub Actions workflow webhook (workflow_run, check_suite).
- GitLab CI webhook (Pipeline Hook).
- Jenkins post-build steps invoking bherald send.
- CircleCI / Drone / Buildkite via their respective webhooks.
- Tekton / Argo Workflows via CloudEvents emitters (native).

Event types:

- digital.vasic.herald.ci.build.queued | started | succeeded | failed | cancelled.
- digital.vasic.herald.ci.test.passed | failed | flaky | skipped.
- digital.vasic.herald.ci.security_scan.finding (Semgrep, Snyk, Trivy, gitleaks).
- digital.vasic.herald.ci.dependency.outdated | vulnerable.
- digital.vasic.herald.ci.coverage.dropped.
- digital.vasic.herald.ci.lint.failed.

Routing logic:

- Failed builds with @-mentions of the author (via subscriber_aliases mapping git-author-email → subscriber).
- Flaky tests batched into a daily digest (per §7.3 throttling).
- Security findings of CRITICAL severity bypass quiet hours.

Subscriber commands:

- **Retry:** — re-trigger the failed build (if integration grants permission).
- **Snooze:** <duration> — silence a flaky test.
- **Triage:** <finding> — mark a security finding for review.

18.5 Deploy Herald (dherald)

Focus: Release / deployment / rollout events.

Event subtree: `digital.vasic.herald.deploy.*`.

Sources:

- ArgoCD / Flux webhook (application synced, health degraded).
- Spinnaker pipeline notifications.
- Kubernetes Event watcher (Deployment rolled out, ReplicaSet failed).
- Custom deploy scripts invoking `dherald send`.

Event types:

- `digital.vasic.herald.deploy.started | succeeded | failed | rolled_back`.
- `digital.vasic.herald.deploy.canary.promoted | canary.aborted`.
- `digital.vasic.herald.deploy.feature_flag.toggled`.
- `digital.vasic.herald.deploy.config.drift_detected`.

Routing:

- Production deploys notify `#prod-deploys` + an Email to `release-eng`.
- Failed deploys with rollback option button (Slack Block Kit Action).

Subscriber commands:

- **Rollback:** `<deploy_id>` — initiate rollback (gated by elevated privilege).
- **Hold:** `<env>` — freeze further deploys to env.
- **Status:** `<env>` — current state of env.

Composes with `rherald` for the full release pipeline.

18.6 Alert Herald (`aherald`)

Focus: Monitoring-alert routing. Sits between alert producers (Alertmanager, Datadog, Grafana) and human/on-call channels (PagerDuty, OpsGenie, Slack). Provides smarter routing than Alertmanager's native receivers.

Event subtree: `digital.vasic.herald.alert.*`.

Sources:

- Prometheus Alertmanager webhook.
- Grafana alert webhook.
- Datadog webhook.
- OpenTelemetry-native alert sources.
- Generic CloudEvents alert producers.

Routing features:

- **De-duplication:** same alert fingerprint within window → single notification.
- **Grouping:** related alerts (same service label + severity) collapse into a digest.
- **Inhibition:** parent alert silences child alerts (per Alertmanager's `inhibit_rules`).
- **Escalation:** integrates with Temporal workflow for time-based escalation chains.

- **Quiet hours** override only for severity=critical + category=incidents.

Subscriber commands:

- Ack:, Silence:, Resolve: (same as sherald).
- Escalate: — bump severity, route to on-call.

18.7 Schedule Herald (scherald)

Focus: Scheduled jobs + recurring notifications (cron-like).

Event subtree: digital.vasic.herald.schedule.*.

Sources:

- River queue periodic jobs.
- scherald serve runs an internal cron-style scheduler.
- External cron jobs invoking scherald send.

Event types:

- digital.vasic.herald.schedule.job.started | succeeded | failed | skipped.
- digital.vasic.herald.schedule.digest.daily | weekly | monthly.
- digital.vasic.herald.schedule.reminder.due (scheduled human reminders).

Built-in digest builders:

- Daily ops digest (yesterday's alerts + deploys + failed builds).
- Weekly project digest (PRs merged, issues opened/closed, contributors).
- Monthly compliance digest (per cherald if installed).

Subscriber commands:

- Snooze: <reminder_id> for <duration>.
- Cancel: <reminder_id>.
- Remind me: <prose> — parse and create a one-shot reminder.

18.8 Incident Herald (iherald)

Focus: Incident lifecycle — declare, escalate, resolve, postmortem.

Event subtree: digital.vasic.herald.incident.*.

Sources:

- PagerDuty / OpsGenie webhooks (incident open/escalated/resolved).
- Internal iherald send from on-call tools.
- Auto-escalation triggered by aherald after N min unacknowledged.

Event types:

- digital.vasic.herald.incident.opened | escalated | resolved | reopened.

- `digital.vasic.herald.incident.commander.assigned`.
- `digital.vasic.herald.incident.update.posted`.
- `digital.vasic.herald.incident.postmortem.due | published`.

Workflow (Temporal):

- On open: page primary on-call → wait 5 min → if no ack, escalate to secondary → wait 10 min → page manager.
- On resolve: schedule postmortem reminder 24h out.
- On postmortem.published: notify stakeholders.

Subscriber commands:

- Page: `<person>` — manual escalation.
- IC: (incident-commander) — assign IC role.
- Update: `<prose>` — post a public update.
- Resolve: — close the incident.

18.9 Release Herald (`rherald`)

Focus: Release lifecycle — tags, changelogs, dependency notifications.

Event subtree: `digital.vasic.herald.release.*`.

Sources:

- Git tag webhooks.
- GoReleaser / Release-Please / semantic-release pipeline hooks.
- Renovate / Dependabot PR webhooks (`dependency.update.available`).
- OCI registry push events (`oci.image.pushed`).

Event types:

- `digital.vasic.herald.release.tagged | published`.
- `digital.vasic.herald.release.changelog.generated`.
- `digital.vasic.herald.release.dependency.update.available | major.update.available`.
- `digital.vasic.herald.release.sbom.generated | provenance.signed`.
- `digital.vasic.herald.release.security_advisory.published` (CVE for a project dep).

Composes with `dherald` — release publish event triggers deploy pipeline notification chain.

Subscriber commands:

- Promote: `<version>` to `<env>`.
- Approve: `<dep_update>` — approve a Renovate PR via authorised channel.

18.10 Compliance Herald (`cherald`)

Focus: Audit + compliance + governance events. For regulated environments (SOC2, HIPAA, PCI-DSS, GDPR).

Event subtree: `digital.vasic.herald.compliance.*`.

Sources:

- Cloud audit logs (AWS CloudTrail, GCP Cloud Audit Logs, Azure Activity Log) via OpenTelemetry / native exporters.
- IAM change webhooks (Okta, Auth0, AzureAD).
- Vault / KMS access logs.
- Database audit log streamers.
- GitGuardian / TruffleHog secret-scan webhooks.

Event types:

- `digital.vasic.herald.compliance.audit.access` (privileged action).
- `digital.vasic.herald.compliance.policy.violation` (e.g. unencrypted bucket, S3 public ACL).
- `digital.vasic.herald.compliance.cert.expiring | expired`.
- `digital.vasic.herald.compliance.license.expiring | non_compliant`.
- `digital.vasic.herald.compliance.access.review.due` (quarterly access reviews).
- `digital.vasic.herald.compliance.secret.exposed`.

Routing:

- All events archived to immutable storage (`docs/herald/audit/` with WORM bit if filesystem supports).
- High-severity events routed to security + legal channels.
- Quarterly summary digest emailed to compliance officer.

Constitution composition: `herald` events plug into the parent project's Constitution §11.4.10 (credentials never tracked) and §11.4.18 (audit-log discipline).

18.11 Future flavors

Identified candidates (NOT in V2 scope; documented to lock-in event-subtree namespaces):

- **fherald** — Finance Herald (billing events, invoice paid/failed, MRR alerts) — `digital.vasic.herald.finance.*`.
 - **mherald** — Marketing Herald (campaign sent, A/B test winner) — `digital.vasic.herald.marketing.*`.
 - **gherald** — Game/Server Herald (multiplayer match events, anti-cheat reports) — `digital.vasic.herald.game.*`.
 - **xherald** — Experiment Herald (feature-flag experiment results) — `digital.vasic.herald.experiment.*`.
 - **hherald** — Hardware Herald (IoT device telemetry alerts) — `digital.vasic.herald.hardware.*`.
 - **lherald** — Legal Herald (contract renewals, NDA expirations) — `digital.vasic.herald.legal.*`.
 - **vherald** — Vendor Herald (third-party service status, SLA breaches) — `digital.vasic.herald.vendor.*`.
-

§19. Diary

docs/herald/diary/main.md (+ main.pdf + main.html) is the **append-only conversation log** of every inbound and outbound message Herald processes.

19.1 Diary path resolution (per R-20)

Resolved relative to the **operator-specified working directory**:

- Standalone Herald: defaults to Herald's own repository root.
- Herald as submodule: defaults to the parent project's root (discovered via the same parent-walk as `find_constitution.sh`).
- Override: `--diary-root <path>` flag, or `[herald].diary_root` in `config.toml`.

19.2 Schema (Markdown)

```
## 2026-05-19T18:30:00Z · pherald · digital.vasic.herald.ci.failed
```

Field	Value
Tenant	<code>`00000000-...-0001`</code>
Event ID	<code>`01931a7c-...-5678`</code>
Source	<code>`github.com/vasic-digital/Herald/actions/runs/4231`</code>
Channels	tgram (delivered), slack (delivered), email (queued)

****Body:****

```
> Build pao u `main`: 3/5 testova nije prošlo. Pogledaj log:  
> https://github.com/vasic-digital/Herald/actions/runs/4231
```

****Subscribers reached:**** alice (tgram + slack), bob (email).

19.3 Sync strategy (per R-03)

- **Pandoc** (Markdown → HTML5) + **WeasyPrint** (HTML → PDF) via Pandoc's `--pdf-engine=weasyprint` flag.
- **Triggers**:
 - Under `<flavor>herald serve`: **fsnotify-watched debounced builds** (500 ms idle, 2 s ceiling). A single editor save fires 4–12 events; debouncing avoids redundant rebuilds.
 - Under one-shot `<flavor>herald send`: post-write hook in the diary writer.
- **Manifest** at `docs/herald/diary/.sync.json` records `{md_sha256, html_sha256, pdf_sha256, source_md_sha256_at_build, built_at}`.
- **Anti-bluff gate** (CM-DIARY-SYNC per Universal §11.4.61):
 1. Read current `main.md` SHA-256.
 2. Assert it equals `source_md_sha256_at_build` in the manifest.
 3. For HTML: re-run Pandoc to a temp file; byte-equality with on-disk HTML.

4. For PDF: extract text (`pdftotext main.pdf -`) and compare to deterministic extraction of the HTML body (byte-equal PDF comparison fails due to embedded timestamps + non-deterministic font subsetting).
-

§20. Extensibility

Per research Topic 6 (operations): two-tier extensibility model.

20.1 Tier A — in-tree adapters (default for V2)

Every channel adapter lives under `commons_messaging/channels/<name>` and implements the `Channel` interface (§11). Out-of-tree authors fork or vendor.

20.2 Tier B — subprocess + manifest (deferred to V2.1)

Spec'd now, implementation deferred. Herald discovers external channel binaries through a manifest file (`heraldchannel.yaml`):

```
name: my-custom-channel
exec: /usr/local/lib/herald/channels/my-channel
supports:
  - text
  - markdown
  - attachments
env:
  required:
    - MY_CHANNEL_TOKEN
protocol_version: 1
```

Communication: stdin/stdout NDJSON with versioned envelope. No in-process linkage — adapters can be written in any language, sandboxed with OS tooling. Mirrors Apprise / Mattermost / git credential-helper.

Explicitly rejected for V2:

- Go plugin stdlib (toolchain-version brittleness, no Windows, breaks `-trimpath`).
 - HashiCorp go-plugin (RPC surface + per-plugin process mgmt — overhead vs. subprocess+JSON).
 - Wazero/WASM (promising; WASI sockets still incomplete; network-bound adapter can't run usefully today). Marked as **V3 roadmap candidate**.
-

§21. Supply chain & release engineering

Per research Topic 5 (operations) + R-18:

21.1 Multi-binary versioning

go.work for local dev (git-ignored). Per-flavor go.mod. Lockstep release: one git tag → GoReleaser builds all flavors + tags commons/vX.Y.Z. Release-Please in manifest mode bumps all modules from Conventional Commits.

21.2 Reproducible builds

Mandatory flags: CGO_ENABLED=0 GOFLAGS=-trimpath. Pinned -ldflags "-buildid= -s -w". Record GOTOOCHAIN.

21.3 SBOMs

Every binary + every container image generates **CycloneDX JSON + SPDX JSON** via syft. Attached as release assets and as OCI referrers on the image.

21.4 Signing

Keyless cosign (Sigstore OIDC via GitHub Actions) for:

- Binaries: cosign sign-blob.
- OCI images: cosign sign.

21.5 SLSA provenance (target: Build L3)

GitHub actions/attest-build-provenance emits **in-toto** statements signed via Sigstore. Move the build into a **reusable workflow** shared across mirrors to reach **SLSA Build L3**.

21.6 Distribution channels

Channel	Tool	Path
Homebrew tap	GoReleaser	vasic-digital/homebrew-herald
Scoop bucket	GoReleaser	vasic-digital/scoop-herald
.deb / .rpm	nFPM	vasic-digital/herald-packages apt+yum repos
Container images	GoReleaser → Docker	GHCR ghcr.io/vasic-digital/ <flavor>herald
Source release	GitHub Releases	Multi-mirror per §103

Verification UX: scripts/verify-release.sh runs cosign verify + cosign verify-attestation against the published cert identity (https://github.com/vasic-digital/Herald/.github/workflows/release.yml@refs/tags/*).

§22. Constitution integration

Once Herald is fully implemented and verified, **promote candidate universal rules** into the root Constitution, AGENTS.md, and CLAUDE.md **via a HelixConstitution PR** (audited per Universal §11.4 + §11.4.17 — universal-vs-project classification), never by editing the parent constitution Submodule from inside Herald (per R-09; see CONSTITUTION_INHERITANCE.md §"Promoting Herald rules into the constitution"). Each Flavor presents its Constitution extensions through the same promotion process.

Note: Herald's implementation **MUST BE** in direct connection with the constitution Submodule — discovery via `find_constitution.sh` parent-walk per §103 / §105 of Herald Constitution.

See [../..guides/HERALD_CONSTITUTION.md](#) and [../..guides/CONSTITUTION_INHERITANCE.md](#).

22.1 How constitution rules are extended via `constitutable/`

A parent project may drop Constitution.md / CLAUDE.md / AGENTS.md (in `constitutable/`, `constitutable/<flavor>/`, `constitutable/<flavor>/<variant>/`, etc.) to layer extensions or overrides on top of the discovered constitution/ Submodule.

Load priority (per `constitutable/` mechanism):

constitution Submodule → `constitutable/**` extensions/overrides for Constitution / CLAUDE.md / AGENTS.md → Project + Submodule definition files.

Each `constitutable/<path>` **MUST** contain at least one of: Constitution.md, CLAUDE.md, or AGENTS.md.

Note: Tests in the constitution Submodule **MUST** be properly extended and updated as `constitutable/` content changes.

Note: Herald is **primarily** consumed as a Submodule of another Project; in that case access to the constitution Submodule is through the root of that project (cloned under `project_root/constitution` or a configured alternative). For **standalone development** of Herald, the constitution is cloned **alongside** Herald (sibling-clone) — current development setup uses this layout. See [../..guides/CONSTITUTION_INHERITANCE.md](#) §"Standalone development" and R-10.

Note: Carefully investigate the codebase of the constitution Submodule before any changes are applied. Comprehensive analysis precedes any extension or promotion.

§23. Specification documents (change rule)

We **MUST** keep the following rule / mandatory constraint in Constitution.md (parent), AGENTS.md, CLAUDE.md, and HERALD_CONSTITUTION.md §106:

IMPORTANT: Whenever this document (docs/specs/mvp/specification.V2.md) or any file under docs/specs/ (any depth) is modified, **comprehensive planning and implementation of all changes is MANDATORY**. This rule does NOT apply to creating or renaming files; for those, explicitly tell the worker (CLI agent) what to do with the new path. Treat every spec edit as a project-wide ripple, not a doc tweak.

Inheritance-gate invariant **I7a-c** enforces presence of this anchor in CLAUDE.md, AGENTS.md, and HERALD_CONSTITUTION.md.

§24. Documentation

README.md MUST be fully updated with all relevant project details. All user guides and manuals are properly linked from README.md, including (when present):

- docs/flavors/<flavor>/ — per-flavor user manual.
- docs/channels/<channel>/ — per-channel setup guide (credentials, webhooks, signing).
- docs/operations/ — deployment + doctor + backups + upgrades.
- docs/security/ — credential handling, signing keys, secret rotation.

We MUST have **mandatory documentation up to the smallest details**: full user guides, manuals, diagrams, schemes in all major formats (Markdown source + PDF + HTML siblings per §11.4.61 + §11.4.65) and other relevant materials.

§25. Testing

Whole project and all derivatives MUST follow testing rules from the root Constitution (Constitution.md), CLAUDE.md, AGENTS.md in the constitution Submodule. Specifically:

- **No bluffing** — every PASS carries positive evidence (§11.4).
- **Paired mutation gates** — every new gate has a paired mutation proving it catches regressions (§1.1).
- **Captured evidence** for test logs (§11.4.2).
- **Test pyramid**: unit tests in each module, integration tests against ephemeral Postgres + Redis (testcontainers-go), end-to-end tests with real-channel sandboxes.
- **No fakes beyond unit tests** (Universal §11.4.27) — integration + end-to-end tests use real Postgres, real Redis, real channel-sandbox or test-bot endpoints.

CI runs the inheritance gate (tests/test_constitution_inheritance.sh) as a precondition to any other test.

§26. Operations

26.1 Deployment

Reference deployment topologies:

- **Single-host Docker Compose** — Herald flavor + Postgres + Redis on one host. Suitable for small teams.
- **Kubernetes Deployment** — Herald flavor as a Deployment, Postgres + Redis as separate StatefulSets (or external managed). HPA on `messaging.client.operation.duration` p95.
- **Serverless / Lambda** — `<flavor>herald send` invoked as a Lambda function (one-shot mode); `serve` mode NOT supported in Lambda due to long-running needs.

26.2 Backups

- Postgres: daily logical backup (`pg_dump`) + WAL archiving for PITR.
- Redis: AOF rewrite + RDB snapshots.
- Diary: standard git history is the backup (everything is in `docs/herald/diary/main.md`).

26.3 Upgrades

- Lockstep across flavors via §21.1 release model.
- Database migrations are **forward-compatible** for 2 minor versions (so a rolling restart works during a deploy).
- `<flavor>herald migrate` is idempotent and safe to run multiple times.

26.4 Operator runbooks

`docs/operations/runbooks/` contains one runbook per common operational scenario:

- "Postgres connection storm" → check `herald_queue_depth` + `pg_stat_activity`.
 - "Telegram bot suspended" → rotate token, update `.env`, run `<flavor>herald doctor`.
 - "Dead-letter spike" → query `dead_letters`, identify pattern, replay or purge.
 - "DKIM verification failing" → DNS check via `<flavor>herald doctor email`.
-

§27. Roadmap

27.1 V2 → V3 candidates

Candidate	Notes
WASM channel adapters (wazero)	Once WASI sockets stabilise — Topic 6 (operations)
LLM-driven message rendering	Auto-summarize long events to channel-appropriate brevity
Slack workflow builder steps	Beyond V2 advanced tier
WhatsApp Business custom-template approval workflow	Once Cloud API matures
Mini-Apps inside Telegram	Subscriber-side UI
Adaptive Card Designer integration for Teams	UX for non-developer template authors
First-class Matrix protocol channel	If demand materializes
Mattermost adapter	Open-source alternative to Slack
Rocket.Chat adapter	Self-hosted alternative

27.2 Deferred V1 Recommendations

V1 R-NN items that V2 did not yet implement (with intended landing):

R	Status in V2	Landing
R-01	✓ Applied (24XXX ports in §9.4)	—
R-02	✓ Applied (single binary, two modes §3.1)	—
R-03	✓ Specified (§19.3)	First implementation cycle
R-04	✓ Applied (§3.3)	—
R-05	✓ Specified (delivery evidence enum §11.13)	First adapter cycle
R-06	✓ Specified (§11.2 Max behind build tag)	When sanctions advisory drafted
R-07	✓ Applied (§7.1)	—
R-08	✓ Applied (§12 ConversationRef)	—
R-09 / R-10 / R-14 / R-15 / R-19 / R-20 / R-21 / R-22	✓ Applied	—
R-11 / R-12	✓ Specified (§9.5)	When first SDK / containers lands
R-13		Next constitution PR

R	Status in V2	Landing
	Deferred — constitutable/ loader + I8/ I9 gates	
R-16	✓ Specified (§15)	First implementation cycle
R-17	✓ Specified (§8.2 algorithm)	First commons_prefix commit
R-18	✓ Specified (§21.1)	First release pipeline

§28. Notes & open questions

- The list of integrated messengers in §11 is the V2 commitment; additional channels may be added in later iterations.
- Whether to ship a **first-party web UI** for subscriber preference management is an open question — current direction is "no UI in V2; expose REST API only; let third parties build UIs".
- Whether aherald should incorporate **machine-learning-based alert grouping** (similar to BigPanda, Moogsoft) is deferred to V3.
- Whether to support **end-to-end encryption** of messages (Signal Protocol via olm/megolm) is deferred — only Matrix bridges currently demand this.
- The **rate-limit cap** for AI-CLI-agent subscribers needs operator-level tuning per deployment to avoid runaway invocation; default suggested 60 messages/minute per agent token.

§29. Changelog (V1 → V2)

V1 → V2 changes (2026-05-19):

- **Renamed** specification.md → specification.V1.md (preserved as historical record).
- **Created** specification.V2.md (this document).
- **New sections** (no V1 counterpart): §2.2 What Herald is NOT, §2.3 Architecture diagram, §4 Event model & wire format (CloudEvents), §5 Architecture overview, §6 Channel addressing & routing (Apprise-style URLs + tags), §7 Subscriber model (identity, preferences, quiet hours, locale), §13 Templating, §14 Localization, §15 Security model (three-layer), §16 Multi-tenancy & isolation (Postgres RLS + Redis ACL), §17 Observability & SLOs (OpenTelemetry), §20 Extensibility (Tier A + Tier B), §21 Supply chain & release engineering (SLSA L3), §26 Operations, §27 Roadmap.
- **Populated V1 TBDs:**
 - System Herald (was TBD) → fully specified §18.3.
 - Project Herald's Constitution rules → folded into §18.2 subscriber-command contract.
 - Input commands → expanded per-flavor in §18.
 - Input attachments → §15.3 content-level scanning + size limits.
 - Others and misc → §18.4–18.11 (7 new flavors + future-flavor namespace reservations).

- **New flavors** beyond V1's pherald + sherald: bherald (Build), dherald (Deploy), aherald (Alert), scherald (Schedule), iherald (Incident), rherald (Release), cherald (Compliance) — plus reserved namespaces for fherald / mherald / gherald / xherald / hherald / lherald / vherald.
- **Game-changing architecture adoptions** (from research):
 - **CloudEvents v1.0** as canonical wire format.
 - **Watermill** as routing core.
 - **River + Postgres** as default queue (transactional enqueue); NATS JetStream as opt-in.
 - **Apprise URL+tag** channel model.
 - **Knock-style preferences** matrix.
 - **OpenTelemetry** end-to-end + Prometheus semantic conventions for messaging.
 - **PostgreSQL RLS** + Redis ACL for multi-tenancy.
 - **SLSA L3** supply chain (cosign + syft SBOM + in-toto provenance).
 - **MJML** for email templates, compiled at author time.
 - **nicksnyder/go-i18n** with CLDR plurals.
- **All V1 text-level Recommendations** (R-01 / R-04 / R-09 / R-10 / R-15 / R-19 / R-20 / R-21 / R-22) carried forward and re-expressed in V2 context.
- **Per-channel feature matrix** (§11.13) — every channel now has documented V1-minimum, V2-advanced, out-of-scope tiers + delivery-evidence ceiling.
- **Email deep dive** (§11.9) — DKIM signing (RFC 6376) + RFC 8058 one-click unsubscribe + DSN bounce parsing + suppression list + doctor_email DNS verification, addressing Gmail/Yahoo 2024 sender requirements.
- **Removed** the V1 Review section — its findings are folded into the V2 body or marked applied/deferred in §27.2. V1's full Review remains in specification.V1.md for traceability.